The temporal knowledge graph & the date model

reconstructing structure, time, and provenance in the Solanus Casey archive

Abstract

Fr. Solanus Casey's archive is 570 letters and 716 notebook pages, and it hides its own structure: a single favor spills across a page break, a date is written once and then omitted for a dozen entries, and the *year* never appears in an entry at all — it sits at the top of the page. This note explains, from first principles, the three `step_7` stages that recover that structure — `stitch_notebooks` (cross-page continuation), `normalize_dates` (the date model: split month/-day + archival year, carry-forward, EDTF, bitemporal), and `build_graph` (a RiC-O temporal knowledge graph with link-prediction) — and the two rules that run through all of them: *every fact cites the exact handwritten region it came from*, and *nothing destructive or billed happens without an explicit, gated opt-in.*

Contents

1 TL;DR

- **The corpus hides its structure.** One favor is split across "Page 1 Cont."; a date is written once and omitted thereafter; the year lives in the page header (`archv_date`), never in the entry. We must *reconstruct* the whole thought and its real date before any search or graph work.
- **Three stages, in dependency order, each a brand-new artifact** (we never edit `documents.json` / `notebooks.json`):
  1. `stitch_notebooks` joins split entries into one **logical entry** while keeping each fragment's exact region box. → `data/stitched_notebooks.json`
  2. `normalize_dates` assembles `month/day` + `archival year` + **carry-forward** into a canonical **EDTF** date, and captures **two** dates per favor (enrolled vs. reported). → `data/dates.json`
  3. `build_graph` builds a **networkx** graph aligned to **RiC-O**, with **provenance** and **EDTF valid-time on every edge**, exports `graph.json` (viz) + `graph.ttl` (RDF), and ships a **KGE link-prediction** scaffold that *proposes* edges into a review queue.
- **Two unbreakable rules.** (1) *Provenance-first*: every fact points back to `doc_id + rid + page + vertices` so the app deep-zooms to the exact ink. (2) *Non-destructive, David-only gold*: rule paths are free and deterministic; every LLM step is gated off by default and cost-logged; suggested links go to a review queue, never the graph.

The receipts rule

An archive that cannot show its receipts is just a rumor with footnotes. So no assertion in this pipeline floats free: each one carries the region id(s) it was read from, and the web app turns those into an IIIF deep-zoom straight to the handwriting. This single constraint shapes every data structure below.

2 Why a graph, and why a date model?

The retrieval layer (embeddings over chunks) is excellent at one question: *"which passages look similar to what I typed?"* But an archivist asks a different *kind* of question — relational and aggregate:

- *Whom did Solanus write to in 1896, and from where?*
- *Which favors reported a cure, and which conditions recur across the Casey family?*
- *Which notebook entries continue onto the next page?*

Those are graph questions: they are about *edges* (wrote-to, has-condition, continues-on) and *time* (in 1896, as-of 1933). A knowledge graph is the natural home. And every one of those questions has a *when* attached, which is why the date model has to come first: an edge without a trustworthy date is half an edge.

3 Stage 1 — `stitch_notebooks`: reconstruct the logical entry

3.1 The problem, concretely

The notebooks are a *running register*. When Solanus ran out of room he simply kept writing on the next page, and the archive labels the spill-over literally. Here is a real page from the corpus (`Volume_3__p002`, pdf page 3), whose first entry is labelled `"Page 1 Cont."`:

> **Page 1 Cont.** — "Mary Briggi - joined her brother V. today – been drinking for five years – Very careless about Church etc. Dec. 14th re- ports very hopeful that his drinking…"

"Page 1 Cont." means *this is still page 1, continued*: the thought that ran off the bottom of the previous page resumes here. If we hand those raw halves to retrieval, the reader — and the embedding model — sees a half-sentence with no date and no context. So before anything else, we rebuild the **logical entry**: the complete unit of meaning, possibly made of several raw fragments.

3.2 The walk, and the cheap signals

We consider joining only the *tail of one page* to the *head of the very next page* — the single place a physical page-break can split a thought. The decision uses transparent, auditable cues:

The continuation signals (additive, not a black box)

signal	what it detects	weight
`page_label =~ /cont/i`	next page literally says "… Cont."	+0.55
numbered chain	"Page N" → "Page N Cont.", N matches	+0.25
truncated tail	prev entry ends mid-clause (no `.` `!` `?` `...` / dash; or `"As-"`)	+0.20
orphan head	next entry opens lower-case or on a connective	+0.15

A page-break with a confidence $\geq$ threshold (default 0.55, so an explicit "Cont." label alone clears the bar) is a candidate join. Grouping into registers is itself non-trivial: the OCR'd notebook header is wildly inconsistent (`"FR. SOLANUS, NOTEBOOK NO. 5."`, `"FR. SOLNAUS, NOTEBOOK NO. S."`, … all the *same* book), so we group on `section` + a *normalized notebook number* rather than the raw string — otherwise one book shatters into 50+ micro-registers and real joins vanish (we measured 154 adjacent page-pairs broken this way).

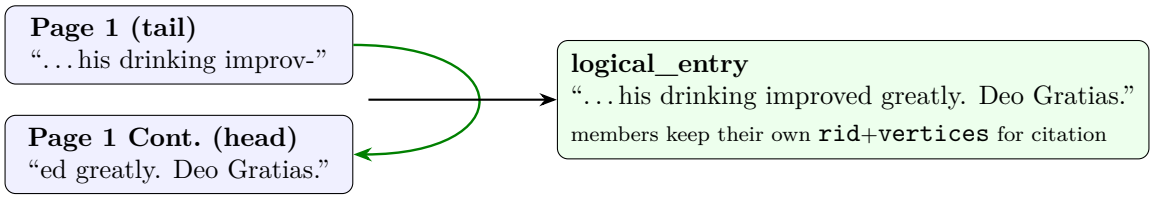
3.3 The one distinction that prevents disaster

Page-level continuation ≠ entry-level continuation

"Page N Cont." tells us the *register* continues onto the next page. It does **not** tell us the last favor on page N is the *same favor* as the first favor on the Cont. page — usually the Cont. page just lists the *next person*. Fusing those would be a **false merge**: two unrelated people's stories welded into one logical entry.

So we **fuse two entries** only when the *thought itself* is split (tail truncated *or* head orphaned) ⇒ `continues_on`/`continued_from` edges. When the label says Cont. but both halves are complete favors, we keep a *page-level* `page_continues_on` edge (the register flows on) and **leave the entries distinct**.

This is the whole pipeline's bias in one line: *we would rather under-stitch (a missing link David can recover later via link-prediction) than over-stitch (a corrupted logical entry).*



3.4 What it writes

`data/stitched_notebooks.json`: `logical_entries` (the whole-favor unit retrieval should embed), `relations` (the KG edges), and an `ambiguous_joins` review queue. Every member fragment retains `rid` + `vertices` + `min_conf`, so a citation still deep-zooms to the precise box even though retrieval now sees the whole entry. That is the "keep both" rule.

The gated multimodal step (no call is made)

`confirm_join_with_vision(...)` would show a vision model the *bottom of page A and the top of page B* and ask whether the thought continues — the 2026 archival-continuation approach, because layout cues (indentation, a bracket, a ditto mark) survive in the image but not in OCR text. It is **gated off**; `run()` makes zero calls. Wiring it is a one-place TODO, and the call would be cost-logged via `lib.providers.llm` → `lib.costlog`.

4 Stage 2 — `normalize_dates`: the date model

4.1 The problem, concretely

An entry usually carries only a month and day ("Mar. 30"), sometimes only a bare day ("22"). The **year almost never appears in the entry**; it lives once, at the top of the page, in `archv_date` (e.g. "1923, November"). And the dates are gloriously human: circa ("c. 1929"), ranges ("c. 1897 to 1904"), feast days, and — crucially for the favors timeline — *two* dates inside one entry.

4.2 Four moves

(1) Hierarchical assembly. Year from the page; month/day from the entry; combine.

(2) Carry-forward. Exactly how a human reads a register: a bare "the 17th" means "the 17th of *whatever month we're in*". A missing month is inherited from the previous entry; a missing year from the page (or the last year resolved on it). Each inference is a small, transparent **confidence** debit — 1.0 means "fully read," lower means "partly inferred."

(3) The year-boundary trick. A single page often straddles New Year, and the archival field records it as month/year *pairs*. A real example:

`archv_date` = `"1923, December  1924, January"`.

A naïve "first year wins" rule dates *every* January entry on that page to **1923** — a guaranteed one-year error. Instead we build a small **month → year** map from the field (Dec → 1923, Jan → 1924)

and pick the year matching the entry's resolved month. A January entry correctly becomes **1924**. The year-boundary page goes from a systematic bug to a correctly *split* timeline.

(4) Bitemporal scan. Favor entries record an **enrollment** date and a later **report/outcome** date. A real example (`Volume_3__p002::doc_1.src_content.10`):

> "Mrs. Catheline M. Sawler **Enrolled May 3rd 1923** — given 3 days to live with pneumonia … **report - Recovered** completely…"

We scan the body just after the cue words (`enroll*`, `report*`/`recovered`) and date each separately, so the enrollment and the outcome carry their *own* valid-times instead of one blurred date. *That* is what makes a real favors timeline possible.

4.3 Why EDTF

Archival dates are fuzzy, and EDTF (Extended Date/Time Format, ISO 8601-2, Library of Congress) lets us store the fuzziness *as data* instead of discarding it or faking precision. We always encode at the precision we actually have — no more:

EDTF, by example (from the Solanus corpus)

corpus date	EDTF	what the notation means
Aug. 23rd, 1896	`1896-08-23`	full date
October 1933, day unknown	`1933-10-XX`	`X` = a known-unknown digit
(only a year survives)	`1929`	"some time in 1929"
c. 1945	`1945~`	`~` = approximate / circa
Feb–Oct 1940	`1940-02/1940-10`	`A/B` = a closed range

4.4 Two code paths

A **pure-Python rule path** (`reconstruct_record`) does the whole job — free, deterministic, no key, no model download — and is what `run()` executes by default, so the temporal layer is reproducible and costs nothing. A real but **gated LLM normalizer** (`llm_normalize_date`) handles the genuinely hard strings (feast days, garbled OCR, prose dates); it is **off** unless `use_llm=True`, capped by `llm_max`, only escalates the *lowest-confidence* records, and is cost-logged on every call.

Output: `data/dates.json`, keyed by record id, each row carrying `raw` + `edtf` + `precision` (`unknown` | `year` | `month` | `da` + `confidence` + `circa` + `method` + the bitemporal `enrolled`/`reported` pair. The raw string sits next to the EDTF value, so every reconstruction is auditable.

5 Stage 3 — `build_graph`: the RiC-O temporal knowledge graph

5.1 One graph, two exports

We build the truth once in **networkx** (a `MultiDiGraph` — *multi* because two entities can be tied by more than one relation; *di* because most relations are directed), then project the same facts into RDF:

- `data/graph.json` — a node/edge list for the web visualization (D3 / Cytoscape).

- `data/graph.ttl` — **RiC-O** RDF (Turtle), so the archive is interoperable Linked Data, not a one-off blob. RiC-O (*Records in Contexts*, the ICA's ontology) is proven on scholarly correspondence. Where RiC-O has a tidy property we use it; where it doesn't, we use a clearly project-namespaced predicate so nothing masquerades as standard vocabulary.

Both exporters read the *same* edge-verb table (`EDGE_KINDS`), so the JSON and the TTL can never drift apart. The verbs: `WROTE_TO`, `LOCATED_AT`, `MEMBER_OF`, `FAMILY`, `ENROLLED`, `PETITIONED_FOR`, `HAS_CONDITION`, `HAS_OUTCOME`, `MENTIONED_IN`, `CONTINUES_ON`. Records become RiC-O `Record`/`RecordPart`; people/orgs/places become `Person`/`CorporateBody`/`Place`; a favor becomes an `Activity` (an enrollment/report is best modeled as an event).

5.2 Where the edges come from — cheapest and most reliable first

Edge sources, ranked by trust

edge	source	`method` tag
`WROTE_TO`, `LOCATED_AT`	letters' separated recipient/sender/location fields	`source`
`MENTIONED_IN` (gold)	`entry.linked` (David's curated connection graph)	`gold_link`
`CONTINUES_ON`	`stitched_notebooks.json` (else `/cont/` fallback)	`stitch`
`ENROLLED`/`HAS_OUTCOME`	`normalize_dates` bitemporal + lexical favor cues	`lexical_cue`
`HAS_CONDITION`	conservative condition cues in entry text	`lexical_cue`

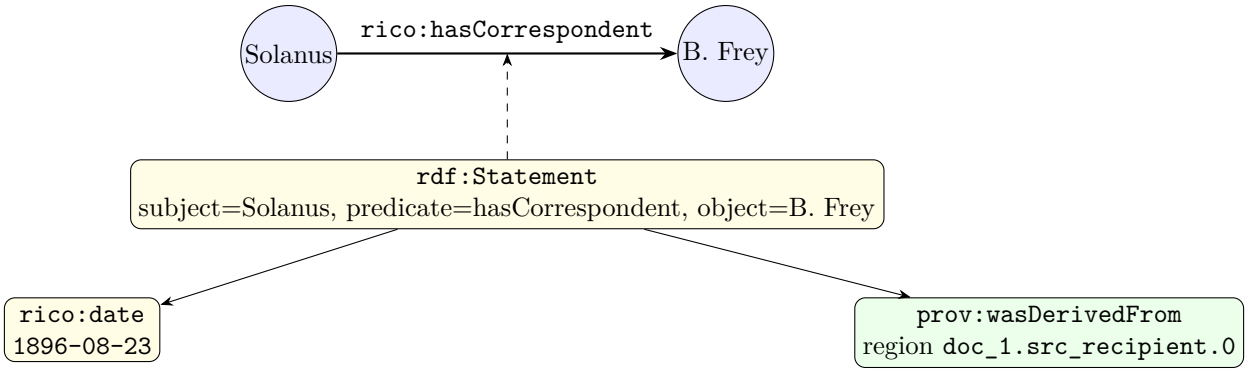
Correspondence is a *free head-start*: recipient, sender, and location are already separated fields per letter. The real letter `Volume_1__p001` ("Very Rev. Bonaventure Frey", "Aug. 23rd, 1896") yields, in one step:

$$\underbrace{\text{Father Solanus Casey}}_{\text{person}} \xrightarrow{\texttt{WROTE\_TO}} \underbrace{\text{Very Rev. Bonaventure Frey}}_{\text{person}}, \quad \texttt{valid\_time} = \texttt{1896-08-23}, \ \texttt{rid} = \texttt{doc\_1.src\_}$$

The lexical favor cues (tagged `lexical_cue`) are deliberately shallow — they exist so the favors backbone is *present* before the proper LLM extractor (`extract_entities`) lands and supersedes them. We would rather miss an edge than assert a wrong one in an archive.

5.3 Two ideas that make the graph trustworthy for an archive

Provenance on every edge (via reification). "Solanus wrote to X *in 1896*, *as evidenced by region* `doc_5.src_recipient.0`" is a statement *about* a statement — provenance and time are metadata *on the edge*, and an RDF edge cannot carry attributes directly. So we **reify**: mint a blank node for the statement and hang the rids off it with PROV-O `prov:wasDerivedFrom`, and the valid-time as a `rico:date` EDTF literal. The receipts stay attached.



Valid-time in EDTF on every edge. Because each edge carries a *when*, the viz can later do "as-of 1933" point-in-time views — show the network exactly as the record stood at a chosen moment.

5.4 Link prediction: a review queue, never an edit

Knowledge graphs are always *incomplete*: a continuation wasn't stitched, two name variants weren't merged, a family tie is implied but never written. A **KG-embedding** model learns a vector $\mathbf{e}_h, \mathbf{e}_t$ per entity and $\mathbf{r}$ per relation so a *true* triple (h, r, t) scores high. DistMult, for instance, scores a triple by a simple three-way inner product,

$$\text{score}(h, r, t) \;=\; \langle \mathbf{e}_h,\ \mathbf{r},\ \mathbf{e}_t \rangle \;=\; \sum_i e_{h,i}\, r_i\, e_{t,i},$$

trained so observed triples outscore randomly corrupted ones. You then score *un-asserted* triples and rank the most plausible missing edges. The stage ships this as a deliberate **scaffold**:

- `train=False` (default) writes an empty-but-valid `data/graph_suggestions.json`, so the review UI always has something well-formed to read.
- `train=True` runs a tiny pure-NumPy DistMult *smoke trainer* (CPU, free) so the control flow is real and runnable, with type constraints so it only proposes *type-sane* edges (a place can't "continue onto" a condition).
- The production path is a clearly-marked **TODO** that plugs in **pykeen** (pulls torch — not yet in the venv).

Suggestions are not assertions

Every proposed edge lands in `graph_suggestions.json` with `status: "proposed"` and the evidence rids that explain *why* it was suggested. David confirms. **Gold is David-only; nothing merges into `graph.json`/`graph.ttl` automatically.** This turns "missing connections" from a dead end into a finite review queue.

6 How to run it

Commands (all FREE by default; nothing is billed)

```
source pipeline_v3/step_7/venv/bin/activate
pip install networkx rdflib          # build_graph's two (lazy) deps

# dependency order:
python stages/stitch_notebooks.py    # -> data/stitched_notebooks.json
python stages/normalize_dates.py     # -> data/dates.json
python stages/build_graph.py         # -> data/graph.json + graph.ttl + suggestions

# smoke tests / inspection:
python stages/stitch_notebooks.py --limit-groups 3
python stages/normalize_dates.py  --limit 50
python stages/normalize_dates.py  --print "Volume_3__p002::doc_1.src_content.10"
python stages/build_graph.py      --kge --top-k 50    # local NumPy KGE smoke
```

Order matters, but absence never crashes. `build_graph` *prefers* `dates.json` and `stitched_notebooks.json`, yet degrades gracefully without them (its own conservative date parser; a `/cont/` heuristic; a record-only graph if `entities.json` is still a stub). Running the stages in order simply makes the graph richer and more accurate. To enable the paid steps *only when David says go*: `normalize_dates.run(use_llm=True, llm_max=<budget>)` and wire the `confirm_join_with_vision` TODO — both auto-cost-logged.

7 Caveats

- **Heavy deps deferred.** `build_graph` needs `networkx` + `rdflib` (imported lazily, so importing the module never explodes); production KGE needs `pykeen`/torch. None are installed yet — the stage stays inspectable and the smoke trainer runs without them.
- **Lexical favor cues are shallow** (substring matches, tagged `lexical_cue`); they seed the favors backbone before the real `extract_entities` LLM stage supersedes them.
- **The rule date parser is conservative** on purpose; the dedicated normalizer is the source of truth, and `build_graph.to_edtf` is only a fallback so the graph has *some* valid-time before `dates.json` exists.
- **We under-stitch deliberately.** A missed join is recoverable later (the KGE review queue); a false merge corrupts a logical entry. The thresholds encode that bias.
- **Prices in `config.py` are placeholders** until verified against each provider's page — but nothing is billed until a gate is flipped.

8 Sources

- **EDTF (ISO 8601-2)** — Library of Congress: https://www.loc.gov/standards/datetime/; validator: https://digital2.library.unt.edu/edtf/.
- **RiC-O (Records in Contexts Ontology)** — International Council on Archives: https://www.ica.org/standards/RiC/ontology.

- **PROV-O (Provenance Ontology)** — W3C: https://www.w3.org/TR/prov-o/.
- **Cross-page continuation via multimodal LLMs** (historical registers): https://arxiv.org/pdf/2512.19675.
- **KG embeddings / link prediction** — completion review: https://www.mdpi.com/2078-2489/13/8/396; **pykeen**: https://github.com/pykeen/pykeen.
- **networkx** https://networkx.org/ **rdflib** https://rdflib.readthedocs.io/.
- Project plan: `pipeline_v3/step_7/RESEARCH_PLAN.md` (PART A: A1 continuation, A2 dates; PART C: temporal KG + KGE).